Lesson 2

Basic Query

School of Media Studies
&Information Technology
HUMBER

Querying Data

- **SELECT Statement Syntax**

The most basic SELECT statement must include at least a SELECT clause and a FROM clause.

Example:

```
USE Database_Name;
GO

SELECT Column_Name, Column_Name
FROM Table_Name;
```

HUMBER School of Media Studies &Information Technology

- Example

```
USE AdventureWorks2022;
GO

SELECT [FirstName], [MIDDLENAME], [LastName]
FROM [Person].[Person];
```

```
USE AdventureWorks2022;
GO

SELECT * FROM [Person].[Person];
```

HUMBER School of Media Studies &Information Technology

- To **exclude** rows that have duplications of the data, you can use **SELECT DISTINCT**
- A WHERE clause can also be added to filter the rows that are returned

```
SELECT DISTINCT JobTitle
FROM AdventureWorks2022.HumanResources.Employee
WHERE OrganizationLevel = 4
```

HUMBER School of Media Studies &Information Technology

Defining a WHERE Clause

- The WHERE clause can include Boolean operators and predicates such as
  - LIKE, BETWEEN,
  - EXISTS, CONTAINS
  - IS NULL, IS NOT NULL
- Boolean operators
  - AND, OR and NOT
- Operators
  - = <> < > >= <=

HUMBER School of Media Studies &Information Technology

Example:

```
USE AdventureWorks2022;
GO
SELECT Name, ProductNumber, StandardCost, ListPrice,
ProductSubcategoryID
FROM Production.Product
WHERE ProductSubcategoryID <> 6 AND ListPrice >= 100
```

Or

```
USE AdventureWorks2022;
GO
SELECT Name, ProductNumber, StandardCost, ListPrice,
ProductSubcategoryID
FROM Production.Product
WHERE ProductSubcategoryID <> 6
AND ListPrice >= 100 AND ListPrice < 200
```

HUMBER School of Media Studies &Information Technology

Inclusive

```
expression [NOT] BETWEEN begin_expression AND
end_expression
```

```
USE AdventureWorks2022;
SELECT Name, ProductNumber, StandardCost, ListPrice
FROM Production.Product
WHERE ProductSubcategoryID <> 6
AND ListPrice BETWEEN 100 and 200
```

Order of Boolean operators

NOT

AND

OR

HUMBER School of Media Studies &Information Technology

Like

- The LIKE clause uses wildcard characters for pattern search
  - **%**: Replaces multiple characters in a string.
    - %at would match at, cat, hat, and that
  - _: Replaces exactly one character in a string.
    - _at would match cat and hat
    - It would **not** match that or at

HUMBER School of Media Studies &Information Technology

- [ ]: Replaces any one character within a set or a range of characters
  - A set is a straight list of characters [abcd]
  - A range is separated by a dash [a–d]

- ^: Any character not within a set or range of characters
  - [^a-d] would be equal to [e–z]

```
USE AdventureWorks2022;
GO
SELECT *
FROM Production.Product
WHERE Name Like N'Flat%';
```

HUMBER School of Media Studies &Information Technology

Manipulating Result Using Aliases

```
select LoginID as [User Login],
HireDate As [Date of Hire],
JobTitle As Position
from [HumanResources].[Employee]
where HireDate < getdate()
order by [Date of Hire],

// OR
  order by HireDate DESC
```

alias

Ascending Sorting

Descending Sorting

HUMBER School of Media Studies &Information Technology

- Manipulating Result Sets Concatenations

```
SELECT [AddressID],
[AddressLine1] + ' ' +
[City] + ' ' +
[PostalCode]
As StreetAddress
FROM [AdventureWorks2022].[Person].[Address]
```

School of Media Studies &Information Technology
HUMBER

Data Types

- The data type for a column is the most fundamental constraint that you can specify for a table
- Data type restricts the range of possible values while defining the maximum amount of space that will be consumed for a column

HUMBER School of Media Studies &Information Technology

SQL Built-In Data Types

Numeric	Character	Dates and Times	Other
Tinyint	Char	Datetime	Binary
Smallint	Nchar	Smalldatetime	Bit
Int	Varchar	Date	Cursor
Bigint	Nvarchar	Datetime2	Xml
Smallmoney	Text	Datetimeoffet	Image
Money	Ntext	Time	Varbinary
Decimal		Timestamp	Uniqueidentifier
Double			Hierarchyid
Float			Rowversion
Real			Geography

HUMBER School of Media Studies &Information Technology

Numeric Data Types

DATA TYPE	RANGE OF VALUES	STORAGE SPACE
TINYINT	0 to 255	1 byte
SMALLINT	-32,768 to 32,767	2 bytes
INT	-2^31 to 2^31-1	4 bytes
BIGINT	-2^63 to 2^63-1	8 bytes
DECIMAL(P,S) and *NUMERIC(P,S)*	-10^38+1 to 10^38-1	5 to 17 bytes
SMALLMONEY	-214,748.3648 to 214,748.3647	4 bytes
MONEY	-922,337,203,685,477.5808 to 922,337,203,685,477.5807	8 bytes
REAL	-3.4e38 to -1.18e38, 0, and 3.4e38 to 1.18e38	4 bytes
FLOAT(N)	-1.79e308 to -2.23e308, 0, 1.79e308 to 2.23e308	4 bytes or 8 bytes

4 bytes if mantissa between 1 and 24, 8 bytes if 25 and 53,

HUMBER School of Media Studies &Information Technology

Decimal and Numeric Data Type Storage

PRECISION	STORAGE SPACE
1 to 9	5 bytes
10 to 19	9 bytes
20 to 28	13 bytes
29 to 38	17 bytes

Decimal and numeric are synonyms and can be used interchangeably.

```
Decimal (10,2) => 12345678.90
```

HUMBER School of Media Studies &Information Technology

Character Data Types

DATA TYPE	STORAGE SPACE
CHAR(n)	Non-Unicode, 1 byte per character defined by *n*, up to a maximum of 8,000 bytes.
VARCHAR(n)	Non-Unicode, 1 byte per character stored up to a maximum of 8,000 bytes
NCHAR(n)	Unicode, 2 bytes per character defined by *n*, up to a maximum of 4,000 bytes
NVARCHAR(n)	Unicode, 2 bytes per character stored up to a maximum of 4,000 bytes

HUMBER School of Media Studies &Information Technology

Character Types

- CHAR (n) and VARCHAR (n):
  - CHAR has fixed length
  - VARCHAR has variable length

- NCHAR(n) and NVARCHAR(n):
  - They store Unicode universel code page (UCS-2) (2 Bytes/char)
  - NCHAR has fixed length
  - NVARCHAR does not have fixed length

HUMBER School of Media Studies &Information Technology

- TEXT and NTEXT
  - These have been depreciated since SQL server 2008
  - NTEXT could store up to 2^30 – 1 chars
  - TEXT could store up to 2^31-1 chars
  - can not use LEN function
  - can not use REPLACE function
  - can not concatenate these
  - CHARINDEX may not work

HUMBER School of Media Studies &Information Technology

- Concatenation
  - + and CONCAT
  - If any of the inputs is NULL, the + operator returns NULL

```
SELECT
[AddressLine1] + N' ' + [City] + N' ' + [PostalCode]
As StreetAddress
FROM [AdventureWorks2022].[Person].[Address]
```

```
SELECT
[AddressLine1] + N' ' + [AddressLine2] + N' ' +
[City] + N' ' + [PostalCode]
As StreetAddress
FROM [AdventureWorks2022].[Person].[Address]
```

```
SELECT
CONCAT([AddressLine1],N' ',[City],N' ',[PostalCode])
As StreetAddress
FROM [AdventureWorks2022].[Person].[Address]
```

HUMBER School of Media Studies &Information Technology

- Substring Extraction and position
  - SUBSTRING (String, From, Length)
  - LEFT (String, Length from left)
  - RIGHT (String, Length from right)

- String Length
  - LEN: Returns no of chars
  - DATALENGTH: Returns data length

HUMBER School of Media Studies &Information Technology

- String alteration
  - REPLACE (String, Substring, New string)
  - REPLICATE (String, No of times)
  - STUFF (String, From, Len, Substitute)

- String Formatting
  - UPPER,
  - LOWER,
  - LTRIM,
  - RTRIM,
  - FORMAT

HUMBER School of Media Studies &Information Technology

Date and Time Data Types

DATA TYPE	RANGE OF VALUES	ACCURACY	STORAGE SPACE
SMALLDATETIME	01/01/1900 to 06/06/2079	1 minute	4 bytes
DATETIME	01/01/1753 to 12/31/9999	0.00333 seconds	8 bytes
DATETIME2	01/01/0001 to 12/31/9999	100 nanoseconds	6 to 8 bytes
DATETIMEOFFSET	01/01/0001 to 12/31/9999	100 nanoseconds	8 to 10 bytes
DATE	01/01/0001 to 12/31/9999	1 day	3 bytes
TIME	00:00:00.0000000 to 23:59:59.9999999	100 nanoseconds	3 to 5 bytes

HUMBER School of Media Studies &Information Technology

Date and Time Functions

- DateTime datatypes
  - GetDate()
  - GetUtcDate()
  - CURRENT_TIMESTAMP
- DateTime2 datatypes
  - SysDateTime()
  - SysDateTimeOffset()
  - SysUtcDateTime()

HUMBER School of Media Studies &Information Technology

GetDate() and Current_TimeStamp produce the same result

```
select
GETDATE(), 'Get Date'
union
select
CURRENT_TIMESTAMP,'Current Time Stamp';
```

	(No column name)	(No column name)
1	2011-10-06 15:33:48.103	Get Date
2	2011-10-06 15:33:48.103	Current Time Stamp

HUMBER School of Media Studies &Information Technology

- DateName function returns the name of the part specified
- DatePart function returns the numeric value of the part specified For example, “1” is returned for January

```
select DateName(WEEKDAY, getdate())
Returns name of current day;

select DatePart(WEEKDAY, getdate())
Returns day of week no (Sunday=1);
```

HUMBER School of Media Studies &Information Technology

- date part:
  - YEAR or YY, or YYYY
  - DAY or DD, WEEKDAY,WD,.....
- The DateAdd and DateDiff functions require a date part to be defined.

```
Select DATEADD(YY, 2, getdate());

Select DATEDIFF(YY, StartDate, EndDate);
```

HUMBER School of Media Studies &Information Technology

```sql
select dateadd(MILLISECOND,-1,'2024-01-25')
select dateadd(MILLISECOND,-2,'2024-01-25')
select dateadd(MILLISECOND,-3,'2024-01-25')
select dateadd(MILLISECOND,-4,'2024-01-25')
select dateadd(MILLISECOND,-5,'2024-01-25')
select dateadd(MILLISECOND,-6,'2024-01-25')
select dateadd(MILLISECOND,-7,'2024-01-25')
select dateadd(MILLISECOND,-8,'2024-01-25')
select dateadd(MILLISECOND,-9,'2024-01-25')
```

1	2024-01-25 00:00:00.000
2	2024-01-24 23:59:59.997
3	2024-01-24 23:59:59.997
4	2024-01-24 23:59:59.997
5	2024-01-24 23:59:59.993
6	2024-01-24 23:59:59.993
7	2024-01-24 23:59:59.993
8	2024-01-24 23:59:59.993
9	2024-01-24 23:59:59.990

HUMBER School of Media Studies &Information Technology

Binary Data Types

DATA TYPE	RANGE OF VALUES	STORAGE SPACE
BIT	Null, 0, and 1	1 bit
BINARY	Fixed-length binary data	Up to 8,000 bytes
VARBINARY	Variable-length binary data	Up to 8,000 bytes

HUMBER School of Media Studies &Information Technology

Data Conversion

- CAST (value AS Type)
- CONVERT (value, type, format)
- PARSE (value AS Type USING culture)
- TRY_CAST (value AS Type)
- TRY_CONVERT (value, type, format)
- TRY_PARSE (value AS Type USING culture)

HUMBER School of Media Studies &Information Technology

CASE Expression

Simple CASE expression:

```
CASE input_expression
     WHEN when_expression THEN result_expression [ ...n ]
     [ ELSE else_result_expression ]
END;
```

```
USE AdventureWorks2022;
GO
SELECT PersonType, FirstName, LastName,
CASE PersonType
      WHEN 'EM' THEN 'Employee'
      WHEN 'SC' THEN 'Sales Contact'
      ELSE 'UNKNOWN'
END As [Person Category]
FROM Person.Person
```

HUMBER School of Media Studies &Information Technology

Searched CASE expression:

```
CASE
     WHEN Boolean_expression THEN
           result_expression [ ...n ]
     [ ELSE else_result_expression ]
END;
```

```
USE AdventureWorks2022;
GO
SELECT ProductId, Name, Weight,
  CASE
    WHEN Weight <= 10 THEN 'Light'
    WHEN Weight > 10 THEN 'Heavy'
    ELSE 'Unknown'
  END AS ProductSize
FROM Production.Product;
```

HUMBER School of Media Studies &Information Technology

Collations

- Collations in SQL Server provide sorting rules, case and accent sensitivity properties for data

- Collations that are used with character data types such as CHAR and VARCHAR dictate the code page and corresponding characters that can be represented for that data type

HUMBER School of Media Studies &Information Technology

- A collation can contain any or all of the following characteristics:
  - Case Sensitive/Insensitive
  - Accent sensitive/Insensitive
  - Kana sensitive/Insensitive
  - Width sensitive/Insensitive

- There are 2,397 variations of collations available in SQL Server 2008, eg
  - Japanese_CI_AI
  - Finnish_Swedish_CS_AI
  - Latin1_General_CI_AS.

HUMBER School of Media Studies &Information Technology

- You can use a different collation in an expression than the one specified on a column when making a comparison

```
. . . WHERE Name = 'Öqvist' COLLATE
Finnish_Swedish_CI_AS
```

- **In the above case SQL Server cannot use an index defined on the column to perform a seek operation**

HUMBER School of Media Studies &Information Technology

User-Defined Data Types

- Custom data types are also known as user-defined data types (UDFs). UDFs are especially useful when you must store the data with the same length or precision over and over again

- To automatically inherit UDF by all new databases create UDF types in the Model system database

HUMBER School of Media Studies &Information Technology

- Use the ALTER TYPE statement to change the definition of your user-defined types

- The DROP TYPE statement should be used to remove the user-defined data types you no longer need in the database

HUMBER School of Media Studies &Information Technology

```
CREATE TYPE [schema_name.]type_name
{ FROM base_type([precision],[scale])
   [NULL | NOT NULL]
}
```

For example

```
CREATE TYPE PersonName
{FROM varchar(50) NOT NULL};
GO
CREATE TABLE TeamMembers
(MemberId int PRIMARY KEY,
MemberName PersonName,
ManagerName PersonName);
GO
```

HUMBER School of Media Studies &Information Technology

Three-Valued Logic

- The predicate can evaluate to:
  - True
  - False
  - Unknown

- The WHERE filter returns only the rows for which the predicate evaluates to true

HUMBER School of Media Studies &Information Technology

```
USE TSQL2012;
GO
SELECT
empid, firstname, lastname, country, region, city
FROM HR.Employees;
```

Results | Messages

	empid	firstname	lastname	country	region	city
1	1	Sara	Davis	USA	WA	Seattle
2	2	Don	Funk	USA	WA	Tacoma
3	3	Judy	Lew	USA	WA	Kirkland
4	4	Yael	Peled	USA	WA	Redmond
5	5	Sven	Buck	UK	NULL	London
6	6	Paul	Suurs	UK	NULL	London
7	7	Russell	King	UK	NULL	London
8	8	Maria	Cameron	USA	WA	Seattle
9	9	Zoya	Dolgopyatova	UK	NULL	London

HUMBER School of Media Studies &Information Technology

```sql
USE TSQL2012;
GO
SELECT empid, firstname, lastname, country,
region, city
FROM HR.Employees
WHERE region = 'WA';
```

Results | Messages

	empid	firstname	lastname	country	region	city
1	1	Sara	Davis	USA	WA	Seattle
2	2	Don	Funk	USA	WA	Tacoma
3	3	Judy	Lew	USA	WA	Kirkland
4	4	Yael	Peled	USA	WA	Redmond
5	8	Maria	Cameron	USA	WA	Seattle

HUMBER School of Media Studies &Information Technology

NULL Regions are not returned

```
USE TSQL2012;
GO
SELECT empid, firstname, lastname, country,
       region, city
FROM HR.Employees
WHERE region <> 'WA';
```

Results | Messages

	empid	firstname	lastname	country	region	city

HUMBER School of Media Studies &Information Technology

```sql
SELECT
empid, firstname, lastname, country, region, city
FROM HR.Employees
WHERE region <> 'WA' OR region IS NULL;
```

Results | Messages

	empid	firstname	lastname	country	region	city
1	5	Sven	Buck	UK	NULL	London
2	6	Paul	Suurs	UK	NULL	London
3	7	Russell	King	UK	NULL	London
4	9	Zoya	Dolgopyatova	UK	NULL	London

HUMBER School of Media Studies &Information Technology

COALESCE and ISNULL

- Return the first not NULL value in their arguments.

```
DECLARE @x AS VARCHAR(4) = NULL;
DECLARE @y AS VARCHAR(10) = 'Something';
DECLARE @z AS VARCHAR(10) = 'Never used';

SELECT COALESCE (@x, @y ,@z) AS COALESCE_RES,
ISNULL(@x, @y) AS ISNULL_RES;
```

Results | Messages

	COALESCE_RES	ISNULL_RES
1	Something	Some

School of Media Studies &Information Technology
HUMBER

Search Arguments (SARG)

- A SARG is a filter expression that is used to limit the number of rows returned by a query <u>and</u> that can use an index seek operation

SARG => Column Operator Expression

Not SARG => `LEFT(Name, 1) = 'A'`

SARG => Name LIKE `'A%'`

HUMBER School of Media Studies &Information Technology

Even thought correct result but not SARG

```
SELECT empid, firstname, lastname, country,
       region, city
FROM HR.Employees
WHERE  ISNULL (region ,'') <> 'WA';
```

Results | Messages

	empid	firstname	lastname	country	region	city
1	5	Sven	Buck	UK	NULL	London
2	6	Paul	Suurs	UK	NULL	London
3	7	Russell	King	UK	NULL	London
4	9	Zoya	Dolgopyatova	UK	NULL	London

HUMBER School of Media Studies &Information Technology

```sql
SELECT orderid, orderdate, empid
FROM Sales.Orders
WHERE shippeddate = @dt;
```

NULL Problem

```sql
SELECT orderid, orderdate, empid
FROM Sales.Orders
WHERE COALESCE(shippeddate, '19000101') =
COALESCE(@dt, '19000101');
```

NOT SARG

```sql
SELECT orderid, orderdate, empid
FROM Sales.Orders
WHERE shippeddate = @dt
   OR (shippeddate IS NULL AND @dt IS NULL);
```

SARG

HUMBER School of Media Studies &Information Technology

Not SARG	SARG
`col LIKE '%ABC%'`	`col LIKE 'ABC'`
`LEFT(col, 3) = 'ABC'`	`LIKE 'ABC%'`

HUMBER School of Media Studies &Information Technology

Character Data Conversion

- SQL Server converts the literal's type to the column's type
- Always use explicit casting

```
use tsql2012
select *
from HR.Employees
where empid = '3';
```

HUMBER School of Media Studies &Information Technology

Both return the same result

```
SELECT empid, firstname, lastname
FROM HR.Employees
WHERE lastname = 'Davis';
```

```
SELECT empid, firstname, lastname
FROM HR.Employees
WHERE lastname = N'Davis';
```

HUMBER School of Media Studies &Information Technology

Filtering Date and Time Data

```
SELECT orderid, orderdate, empid, custid
FROM Sales.Orders
WHERE orderdate = '02/12/07';
```

Ambiguous

```
SELECT orderid, orderdate, empid, custid
FROM Sales.Orders
WHERE orderdate = '20070212';
```

Language Neutral

```
SELECT orderid, orderdate, empid, custid
FROM Sales.Orders
WHERE orderdate = '2007-02-12';
```

DATETIME2

HUMBER School of Media Studies &Information Technology

Making SARG date filtering

```
SELECT orderid, orderdate, empid, custid
FROM Sales.Orders
WHERE YEAR(orderdate) = 2007 AND
      MONTH(orderdate) = 2;
```

Not SARG

```
SELECT orderid, orderdate, empid, custid
FROM Sales.Orders
WHERE orderdate >= '20070201' AND
      orderdate < '20070301';
```

SARG

```
SELECT orderid, orderdate, empid, custid
FROM Sales.Orders
WHERE orderdate BETWEEN '20070201'
      AND '20070228 23:59:59.999'
```

Unexpected Result

HUMBER School of Media Studies &Information Technology

Sorting data

```
SELECT
        empid, firstname, lastname, city,
        MONTH(birthdate) AS birthmonth
FROM
        HR.Employees
WHERE
        country = N'USA' AND region = N'WA'
ORDER BY
        city;
-- ORDER BY 4;
```

HUMBER School of Media Studies &Information Technology

- When DISTINCT clause is used, you are limited in the ORDER BY list to only elements that appear in the SELECT list

```
SELECT DISTINCT
      city
FROM
      HR.Employees
WHERE
      country = N'USA' AND region = N'WA'
ORDER BY
      city;
```

HUMBER School of Media Studies &Information Technology

- Aliases may be used in ORDER BY clause

```
SELECT
      empid, firstname, lastname, city,
      MONTH(birthdate) AS birthmonth
FROM
      HR.Employees
WHERE
      country = N'USA' AND region = N'WA'
ORDER BY
      birthmonth;
```

HUMBER School of Media Studies &Information Technology

Ordering NULLs

- SQL Server the sorts NULLs before non-NULLs when using an ascending direction

```
SELECT orderid, shippeddate
FROM Sales.Orders
WHERE custid = 20
ORDER BY shippeddate;
```

Results | Messages

	orderid	shippeddate
1	11008	NULL
2	11072	NULL
3	10258	2006-07-23 00:00:00.000
4	10263	2006-07-31 00:00:00.000
5	10351	2006-11-20 00:00:00.000
6	10368	2006-12-02 00:00:00.000
7	10382	2006-12-16 00:00:00.000

HUMBER School of Media Studies &Information Technology

TOP clause

- You can limit filter to a requested number or percent of rows from the query result based on indicated ordering .

```
SELECT TOP (3) orderid, orderdate, custid, empid
FROM Sales.Orders
ORDER BY orderdate DESC;
```

Results | Messages

	orderid	orderdate	custid	empid
1	11077	2008-05-06 00:00:00.000	65	1
2	11076	2008-05-06 00:00:00.000	9	4
3	11075	2008-05-06 00:00:00.000	68	8

Non deterministic

HUMBER School of Media Studies &Information Technology

- With Ties clause

```
SELECT TOP (3) WITH TIES orderid,
orderdate, custid, empid
FROM Sales.Orders
ORDER BY orderdate DESC;
```

Deterministic

HUMBER School of Media Studies &Information Technology

- The PERCENT option rounds up the resulting number of rows if it's not whole.

```
SELECT TOP (1.1) PERCENT orderid, orderdate,
   custid, empid
FROM Sales.Orders
ORDER BY orderdate DESC;
```

HUMBER School of Media Studies &Information Technology

OFFSET-FETCH

- The OFFSET-FETCH option is a filtering option that can be used to filter data based on a specified number of rows and ordering
- OFFSET-FETCH option is standard SQL while TOP is not standard SQL

```
SELECT orderid, orderdate, custid, empid
FROM Sales.Orders
ORDER BY orderdate DESC, orderid DESC
OFFSET 50 ROWS FETCH NEXT 25 ROWS ONLY;
```

HUMBER School of Media Studies &Information Technology

- The OFFSET clause doesn't require a FETCH clause

```
SELECT orderid, orderdate, custid, empid
FROM Sales.Orders
ORDER BY orderdate DESC, orderid DESC
OFFSET 50 ROWS;
```

HUMBER School of Media Studies &Information Technology